

LIBRARY MANAGEMENT SYSTEM

Authentication Module — Best Django Solution

Project component covered: Registration, login templates, logout, user profiles, and role-based permissions using Django's built-in authentication.

1. Recommended Architecture

Use Django's built-in User model for authentication and a one-to-one Profile model for roles. This is preferable to creating a custom password system because Django handles password hashing, sessions, authentication, and security features.

```
library_management/
├── manage.py
├── library_management/
│   ├── settings.py
│   ├── urls.py
│   └── ...
├── accounts/
│   ├── migrations/
│   ├── templates/accounts/
│   │   ├── login.html
│   │   ├── register.html
│   │   └── dashboard.html
│   └── ...
├── admin.py
├── apps.py
├── decorators.py
├── forms.py
├── models.py
├── signals.py
├── urls.py
├── views.py
└── library/
```

2. User Roles

Role	Permissions
Admin	Full system access; manage users, books, borrowing and reports.
Librarian	Manage books, issue/return books and view borrowing records.
Student	Search/view books, borrow books and view personal borrowing records.

3. Install and Create the App

```
python manage.py startapp accounts
```

Add `accounts.apps.AccountsConfig` to `INSTALLED_APPS` in `settings.py`.

4. User Profile and Roles

accounts/models.py

```
from django.db import models
from django.contrib.auth.models import User

class Profile(models.Model):
    ROLE_CHOICES = (
        ('ADMIN', 'Admin'),
        ('LIBRARIAN', 'Librarian'),
        ('STUDENT', 'Student'),
    )

    user = models.OneToOneField(User, on_delete=models.CASCADE)
    role = models.CharField(
        max_length=20,
        choices=ROLE_CHOICES,
```

```

        default='STUDENT'
    )

    def __str__(self):
        return f'{self.user.username} - {self.role}'

```

5. Automatically Create a Profile

accounts/signals.py

```

from django.db.models.signals import post_save
from django.contrib.auth.models import User
from django.dispatch import receiver
from .models import Profile

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

```

accounts/apps.py

```

from django.apps import AppConfig

class AccountsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'accounts'

    def ready(self):
        import accounts.signals

```

6. Secure Registration Form

Students should register only as STUDENT. Admin creates Librarian accounts. This prevents a new user from registering as an administrator.

accounts/forms.py

```

from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm

class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = [
            'username', 'first_name', 'last_name',
            'email', 'password1', 'password2'
        ]

    def save(self, commit=True):
        user = super().save(commit=False)
        user.email = self.cleaned_data['email']
        user.first_name = self.cleaned_data['first_name']
        user.last_name = self.cleaned_data['last_name']

        if commit:
            user.save()

        return user

```

7. Authentication Views

accounts/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from .forms import RegisterForm

def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            user.profile.role = 'STUDENT'
            user.profile.save()
            messages.success(
                request,
                'Registration successful. You can now log in.'
            )
            return redirect('login')
    else:
        form = RegisterForm()

    return render(
        request,
        'accounts/register.html',
        {'form': form}
    )

def user_login(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(
            request,
            username=username,
            password=password
        )

        if user is not None:
            login(request, user)
            return redirect('dashboard')

        messages.error(request, 'Invalid username or password.')

    return render(request, 'accounts/login.html')

@login_required
def user_logout(request):
    logout(request)
    return redirect('login')

@login_required
def dashboard(request):
    role = request.user.profile.role

    if role == 'ADMIN':
        template = 'accounts/admin_dashboard.html'
    elif role == 'LIBRARIAN':
        template = 'accounts/librarian_dashboard.html'
    else:
        template = 'accounts/student_dashboard.html'

    return render(request, template)
```

8. Role-Based Access Control

accounts/decorators.py

```
from django.shortcuts import redirect
from django.contrib import messages

def role_required(allowed_roles):
    def decorator(view_func):
        def wrapper(request, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect('login')
```

```

        if request.user.profile.role not in allowed_roles:
            messages.error(
                request,
                'You do not have permission to access this page.'
            )
            return redirect('dashboard')

        return view_func(request, *args, **kwargs)

    return wrapper
return decorator

```

Example: librarian/admin book management

```

from django.contrib.auth.decorators import login_required
from accounts.decorators import role_required

@login_required
@role_required(['LIBRARIAN', 'ADMIN'])
def manage_books(request):
    return render(request, 'library/manage_books.html')

```

Example: admin-only user management

```

@login_required
@role_required(['ADMIN'])
def manage_users(request):
    return render(request, 'accounts/manage_users.html')

```

9. URLs

accounts/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register, name='register'),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
]

```

library_management/urls.py

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('accounts.urls')),
]

```

10. Login Template

```

<!DOCTYPE html>
<html>
<head>
    <title>Library Management System - Login</title>
</head>
<body>
    <h2>Library Management System</h2>
    <h3>Login</h3>

    {% if messages %}
        {% for message in messages %}
            <p>{{ message }}</p>
        {% endfor %}
    {% endif %}

    <form method="POST">
        {% csrf_token %}
        <label>Username</label>
        <input type="text" name="username" required>

        <label>Password</label>
        <input type="password" name="password" required>

        <button type="submit">Login</button>
    </form>

```

```

        <a href="{% url 'register' %}">Create an account</a>
    </body>
</html>

```

11. Registration Template

```

<!DOCTYPE html>
<html>
<head>
    <title>Register - Library Management System</title>
</head>
<body>
    <h2>Create Student Account</h2>

    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Register</button>
    </form>

    <a href="{% url 'login' %}">Already have an account? Login</a>
</body>
</html>

```

12. Database and Initial Setup

```

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

```

13. Admin Configuration

accounts/admin.py

```

from django.contrib import admin
from .models import Profile

@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'role')
    list_filter = ('role',)

```

14. Security and Quality Requirements

- Use Django's built-in authentication rather than storing passwords manually.
- Never store plain-text passwords.
- Use {% csrf_token %} in every POST form.
- Require authentication before accessing protected pages.
- Restrict Admin and Librarian roles to authorized users.
- Do not allow public registration as Admin or Librarian.
- Use role checks on the server side; hiding a button alone is not security.
- Use Django messages for clear login and permission error feedback.

15. Final Workflow

```

New user
↓
Student registration
↓
Django validates and hashes password
↓
Profile created automatically
↓
User logs in
↓
Django authenticate() verifies credentials
↓

```

Django creates authenticated session
↓
Role is checked
 ■■■ ADMIN → Admin Dashboard
 ■■■ LIBRARIAN → Librarian Dashboard
 ■■■ STUDENT → Student Dashboard
↓
Protected library functions become available
↓
Logout → session terminated

16. Expected Result

The completed module provides a secure and professional authentication layer for the Library Management System. Students can register and log in, while administrators control privileged accounts and librarians can perform authorized library operations. Unauthorized users are blocked even if they attempt to access protected URLs directly.

Recommended project demonstration

During your presentation, demonstrate three accounts: a Student, a Librarian, and an Admin. Log in with each account and show that the dashboard and accessible functions change according to the role. Then attempt to access a restricted URL as a Student to demonstrate that server-side permission checking works.